

THE INTERNET OF THINGS



News, views and development on the Internet of Things

“Engineers working on IoT devices have to expand their thinking to the whole system, even if they are not building it all”

Jim Tung, fellow, MathWorks Inc., speaks to Janani Gopalakrishnan Vikram about the complexity posed by the Internet of Things (IoT), the mindset change required for engineers to handle it with sophistication, and how model-based design can help in this

JANANI GOPALAKRISHNAN VIKRAM

What kind of complexity does the IoT pose to engineers designing devices for this era?

The IoT is an ecosystem of technologies working together to create dynamic systems of physical devices and information. These systems may do several things like providing data, aggregating data provided by multiple systems, analysing the aggregated data to derive useful information and trends, presenting the information and trends to users in a timely manner on a range of platforms, including Web, mobile devices and desktops/laptops, and changing the system's own behaviour based on the derived information and trends.

So the complexity of the IoT can be seen in flexible and smart embedded systems, and systems of systems, that collect data and often solve specific problems locally; or in algorithms needed to extract insight from the data and build systems that operate and adapt autonomously (these systems may combine feedback control, computer vision and machine learning, for example, and may include algorithms not yet imagined); and in deploying algorithms quickly to where the data resides so that analysis takes place as efficiently and quickly as possible,

whether the embedded processors and FPGAs are in a local node, multi-core server farms or the cloud, perhaps using computational frameworks such as Hadoop.

What mindset change is required for engineers to handle such complexity in a more efficient way, without getting flustered?

Engineers working on these systems of systems, or any of their components, should understand—at least at a high level—the requirements of the system as a whole and the capabilities of each of the subsystems. Creating an embedded system for a local device, for example, is only one part of designing an IoT system. If the engineers don't understand how the data generated by it might be used, they might design an embedded system optimised to a local problem, rather than the whole challenge. They have to expand their thinking to the whole system, even if they are not building it all.

Automating development tasks such as code generation for embedded systems or deploying algorithms into a production IT infrastructure can also reduce complexity in key areas. The technology is well-proven, but often requires a change in mindset to take advantage of it.

How can model-based design help here?

Models enable an engineer to think at a higher level of abstraction, which, in turn, makes it easier to conceptualise the entire system. Simulating those models enables the engineer to evaluate and improve the system's behaviour across a wide range of scenarios, including those that are difficult to replicate in real life. Automatic code generation means fast and robust implementation to devices ranging from FPGAs and microcontrollers, to SoC devices such as the Xilinx Zynq, to rugged industrial programmable logic controllers.

Model-based design (MBD) can be done using tools like Simulink, Stateflow and MATLAB. MATLAB and Simulink can also connect directly to popular low-cost and open-source hardware such as Arduino and Raspberry Pi, without the need for a code generator. Tethering and targeting the hardware support packages enables makers and developers to create devices of the future with today's tools.

Since MBD provides capabilities for simulating the behaviour of the physical system, and not just the algorithms, it is very useful for developing algorithms that require comparison to a reference model to assess the behaviour of the system and detect anomalies.